

Repository-wise C++ code to Java Translation leveraging the LLM

Anonymous Author(s)*

ABSTRACT

Code translation—converting code from one programming language to another while preserving functionality—is critical for legacy system migration, cross-platform development, and ecosystem modernization. Traditional rule-based or manual approaches are limited in scope and efficiency. While large language models (LLMs) have shown promise in code generation, most existing research focuses on translating isolated methods or small files, leaving project-level translation—where accumulated language paradigm differences become significant—largely unexplored.

This paper proposes TOOL, a hierarchical, macro-to-micro translation framework designed to mitigate error propagation in large-scale code translation. The framework first performs static analysis to extract call graphs and decompose code into functional modules using a custom Java parser tool. Translation then proceeds in two layers: (1) at the class level, where overall architecture and header files are designed with language-specific pattern mapping, and (2) at the method level, where implementations are translated bottom-up along call chains to ensure dependency stability. To bridge library and runtime differences, the framework dynamically generates necessary dependencies and an autonomous agent iteratively compiles, tests, and repairs the translated code using intelligent error analysis and fix strategies.

We evaluate TOOL by translating several cohesive Java modules to C++, two languages with significant differences in underlying execution models. Experimental results demonstrate that ... **TODO: Add experimental results**

Keywords: Code translation, Large language models, Project-level translation, Java, C++, Static analysis, Hierarchical translation

1 INTRODUCTION

The increasing need for software migration, cross-language interoperability, and multi-platform deployment has made automated code translation a valuable research area. Early techniques relied on handcrafted rules or template-based transformations, which are difficult to scale. Recent advances in LLMs have enabled more flexible and context-aware code generation, yet their application in translation remains largely confined to small, self-contained code snippets. Translating entire projects or interconnected modules introduces challenges such as architectural mismatch, dependency management, and compounding semantic gaps between languages—especially those with different runtime paradigms like Java (managed, JVM-based) and C++ (unmanaged, native).

This paper addresses the problem of error accumulation in project-level translation. We argue that translating files or methods independently without a high-level understanding of

the module’s purpose leads to irreconcilable inconsistencies. Instead, we propose a top-down understanding followed by bottom-up translation, where the LLM first comprehends the macro-structure of a functional module, plans the overall translation strategy, and then executes translation in a layered manner.

Our main contributions are:

- **A hierarchical translation framework (tool)** that separates class-level design from method-level implementation, enabling macro-to-micro translation with reduced error propagation.
- **A call-graph-guided, bottom-up translation order** that stabilizes dependencies by translating leaf methods first, ensuring callees are available when translating callers.
- **An autonomous compilation and repair agent** that validates and corrects the translated code using intelligent error analysis, supporting iterative compilation with automated fix strategies.

2 RELATED WORK

2.1 Traditional Code Translation Methods

Early work recognized that programs possess rich syntactic structures and that direct application of sequence-to-sequence models often yields syntactically invalid outputs. To address this, Chen et al. [1] pioneered the use of tree-to-tree neural networks, which encode and decode abstract syntax trees (ASTs) to explicitly model program structure. This was further refined by Drissi et al. [3], who introduced a grammar-driven tree-to-tree model that hard-codes target language syntax rules into the decoder, guaranteeing grammatical correctness. These studies established the importance of structural awareness in code translation but were limited by their dependence on parsers, manually defined grammars, and parallel tree-aligned corpora, which hindered generalization and scalability.

To overcome the scarcity of parallel data, The field shifted toward data-driven, unsupervised methods. Roziere et al. [17] introduced TransCoder, which leverages back-translation and denoising auto-encoding on monolingual code corpora to learn cross-lingual code representations without supervision. Concurrently, pre-trained models such as CodeBERT and CodeT5 adapted the pre-training paradigm from natural language processing to code, learning general-purpose representations via masked language modeling. These approaches demonstrated the power of scale and transfer learning, moving the focus from designing task-specific architectures to fine-tuning general-purpose models. However, Malyala et al. [11] found that purely data-driven models often exhibit systematic errors in complex translation scenarios, such as type

conversions and loop restructuring. Pan et al.[14] conducted a large-scale empirical study showing that LLMs introduce a significant number of bugs during translation, particularly in real-world projects, highlighting the limitations of statistical approaches in understanding deep semantic and syntactic constraints.

2.2 LLM Based Code Translation Methods

To mitigate the shortcomings of purely data-driven models, recent research emphasizes augmenting LLMs with programming language-specific knowledge, such as type systems, safety rules, and API mappings. This is especially critical for security-sensitive migrations, such as from C to Rust. Luo et al. [10] proposed the IRENE framework, which combines static analysis to extract “high-risk patterns” with retrieval-augmented generation to guide the model toward producing Rust code that complies with ownership rules. Similarly, Hong et al. [4] focused on type migration, employing a call-graph-based strategy to coordinate interface changes across functions. These hybrid systems effectively address the “semantic drift” and safety violations common in naive LLM translations. Extending this neuro-symbolic approach, Ibrahimzada et al. [5] introduced AlphaTrans, which compositionally combines symbolic knowledge with neural models for repository-level translation and validation. Meanwhile, Nitin et al.[12] enhances translation by generating multi-modal specifications (e.g., comments, tests) to provide additional context. While these methods significantly improve reliability, they often require substantial engineering effort to construct and maintain domain-specific knowledge bases.

2.3 Hallucination in General-Purpose LLMs

Hallucination in large language models (LLMs) refers to the generation of factually incorrect or contextually inconsistent content. Early research focused on text-based hallucinations, with recent work expanding to multimodal settings [15]. Detection methods have evolved from lexical metrics (ROUGE, BLEU) to more robust frameworks incorporating multi-dimensional evaluation [2] and domain-specific approaches like self-reflection in medical QA [6]. Mitigation strategies include knowledge augmentation, prompt engineering, and adversarial defenses [16, 19].

Code generation introduces unique challenges due to strict syntactic and functional requirements. Liu et al. [8] identified three primary hallucination types: Requirement Conflict, Knowledge Hallucination, and Code Inconsistency. In repository-level generation, Zhang et al.[22] further categorized these into Task Requirement Conflicts, Factual Knowledge Conflicts, and Project Context Conflicts.

The causes are multifaceted, including training data quality issues, intent comprehension limitations, and inadequate context awareness [22]. Impacts are severe: 95.30% of hallucinations cause functional errors, with hallucinated code having only 6.26% pass@1 rate versus 54.73% for clean code [8].

Retrieval-augmented generation (RAG) has proven effective, improving pass@1 scores across models (e.g., +4.38% for ChatGPT) [22]. Other approaches include self-refinement prompts, chain-of-thought reasoning, and specialized code models.

2.4 Repo-level Code Translation Methods

As model capabilities have grown, the research scope has expanded from function-level snippets to more complex, practical scenarios. Repository-level code translation presents unique challenges, including managing inter-file dependencies, build configurations, and project-wide context. Wang et al. [18] introduced RepoTransBench, a large-scale, multi-language benchmark for evaluating repository-level translation, revealing that even state-of-the-art LLMs struggle (with success rates around 32.8%). To tackle this, Zhang et al. [21] proposed a skeleton-guided translation framework that first extracts a high-level structural skeleton in the target language before filling in implementation details. Yuan et al.[20] and Ou et al.[13] further explored project-level translation by synergistically integrating knowledge graphs and LLMs, or by evolving a triple knowledge-augmented LLM within the repository context.

Parallel to scaling up, improving the human-in-the-loop experience has become a key focus. Liu et al.[9] developed hmCodeTrans, an interactive translation framework that allows developers to provide real-time feedback. By employing attention caching and suffix splicing, the system achieves millisecond-level responsiveness, reducing human effort by approximately 65% compared to traditional post-editing workflows. This marks a shift from pursuing full automation toward designing collaborative tools that integrate seamlessly into developer workflows.

Our work is situated at this intersection. We aim to develop a translation framework that leverages the generative strength of LLMs while deeply integrating extensible, domain-aware knowledge and validation mechanisms. We target translation tasks involving complex algorithm logic and API migrations (akin to Type 3/4 in the taxonomy by Jiao et al.[7]). Inspired by hybrid systems like IRENE [10] and compositional approaches like AlphaTrans [5], we propose a more generalized and extensible translation knowledge orchestration method. This approach is designed to dynamically retrieve and apply relevant constraints and patterns, ensuring high semantic fidelity and project-level usability without sacrificing flexibility.

3 APPROACH

This section presents the detailed design of TOOL, our hierarchical framework for project-level Java to C++ code translation. We first provide an overview of the framework architecture, then describe each component in detail.

3.1 Framework Overview

Figure 1 illustrates the overall architecture of TOOL. The framework adopts a **macro-to-micro translation strategy** that

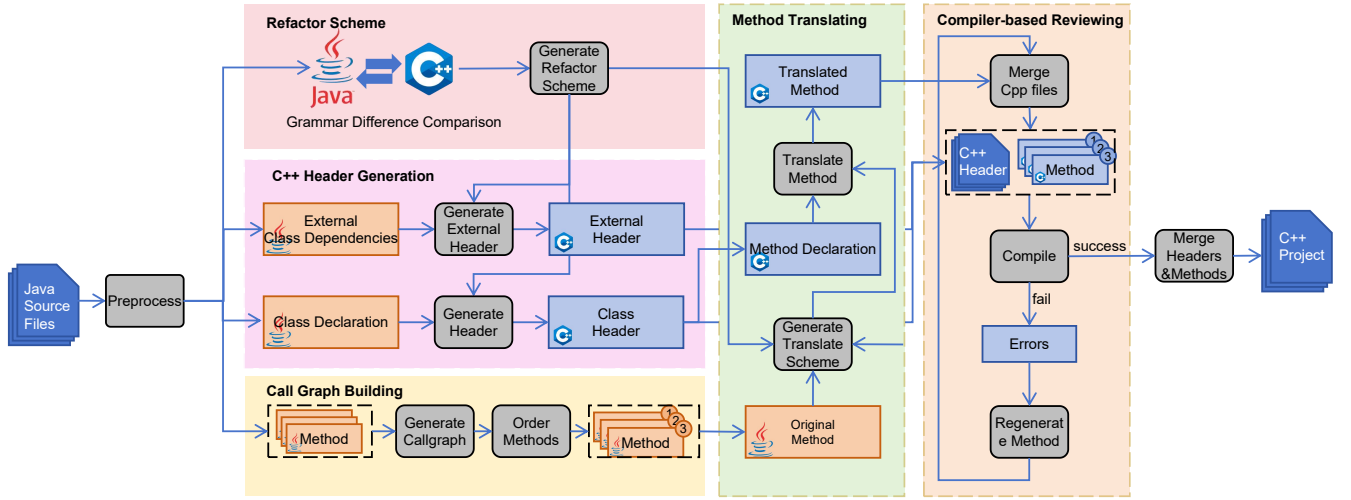


Figure 1: Overall architecture of tool showing the three-phase translation process.

separates high-level structural design from low-level implementation details. This hierarchical decomposition addresses two key challenges in project-level code translation: (1) managing complexity at scale, and (2) mitigating error propagation through context-aware translation ordering.

The translation process consists of three main phases:

- (1) **Static Analysis Phase:** A custom Java parser tool extracts the project structure, including class hierarchies, method signatures, and call graph dependencies. This phase decomposes the codebase into a hierarchical graph representation comprising *Header nodes* (classes, interfaces, enums) and *Method nodes* (individual method implementations). The graph captures both structural relationships (inheritance, type dependencies) and behavioral relationships (method call dependencies), which guide subsequent translation ordering.
- (2) **Hierarchical Translation Phase:** The framework performs translation in two interdependent layers:
 - *Header-level translation:* For each Java class, interface, and enum, the LLM generates a translation scheme that maps Java language constructs to appropriate C++ patterns (e.g., interfaces to abstract base classes with virtual functions, checked exceptions to error code handling). This layer establishes the structural foundation and type mapping conventions for the entire project.
 - *Method-level translation:* Methods are translated following a bottom-up ordering derived from the call graph. Callee methods are translated before their callers, enabling *context propagation*—already-translated code serves as reference for translating dependent methods. This dependency-aware approach significantly reduces translation inconsistencies and API mismatches.

- (3) **Validation and Repair Phase:** An autonomous compilation agent iteratively compiles the translated C++ code, analyzes compilation errors, and applies fixes using LLM-generated suggestions. The agent operates through a tool-use interface, invoking compilation and file modification primitives to iteratively resolve errors until compilation succeeds or maximum attempts are reached.

3.2 Static Analysis and Graph Construction

The first phase of TOOL involves parsing the Java source code and extracting structural information necessary for guiding the translation process. This phase constructs a project-level dependency graph that serves as the foundation for all subsequent translation decisions.

3.2.1 Project Graph Model. The static analysis phase constructs a bipartite graph representation consisting of two types of nodes:

- **Header Nodes:** Represent type declarations (classes, interfaces, enums). Each Header node captures:
 - The complete type declaration and field members
 - Inheritance relationships (parent class, implemented interfaces)
 - Import dependencies (external types required by this compilation unit)
 - A reference to all contained Method nodes
- **Method Nodes:** Represent individual method implementations. Each Method node captures:
 - The method signature (return type, parameter types, modifiers)
 - The complete method body implementation
 - Call dependencies (**children:** methods called by this method; **parents:** methods that call this method)

- External call references (calls to methods outside the project boundary)

This bipartite structure enables separate handling of structural translation (Headers) and behavioral translation (Methods), while maintaining explicit relationships between them.

3.2.2 Call Graph Extraction. To establish the correct translation order for methods, the framework extracts the complete call graph from the Java source code. We integrate **java-callgraph2**, a static analysis tool that identifies method call relationships through points-to analysis.

The call graph construction process involves:

- (1) Parsing method call traces from **java-callgraph2** output
- (2) Matching caller and callee signatures to Method nodes in the project graph
- (3) Establishing bidirectional edges between caller-callee pairs
- (4) Identifying external dependencies (calls to library methods outside the project)

The resulting call graph enables topological sorting of methods into translation layers, where all dependencies of a method in layer n are guaranteed to be in layers 1 through $n - 1$.

3.3 Hierarchical Translation Pipeline

The translation pipeline operates on the project graph in two distinct phases: Header-level translation establishes the structural foundation and type conventions, followed by Method-level translation that leverages dependency ordering for context propagation. Each phase employs a dual-pass strategy consisting of scheme generation followed by code generation.

3.3.1 Header-Level Translation. Header-level translation addresses the structural differences between Java and C++, establishing consistent conventions for the entire project before translating individual method implementations.

Scheme Generation. For each Header node, the LLM generates a **translation scheme**—a structured document that specifies:

- **Type mapping strategy:** How Java types map to C++ types (e.g., `ArrayList<T>` $\rightarrow$ `std::vector<T>`, `String` $\rightarrow$ `std::string`, `HashMap<K,V>` $\rightarrow$ `std::unordered_map<K,V>`)
- **External dependency handling:** Identification of external dependencies that cannot be directly translated (e.g., third-party libraries, Java framework types), with strategies for mocking, replacing, or omitting them
- **Memory management strategy:** How to replace Java’s garbage collection with appropriate C++ memory management (e.g., using stack allocation where possible, smart pointers for heap-allocated objects, RAII patterns for resource management)

- **Inheritance mapping:** How to map Java interfaces and abstract classes to C++ (e.g., interfaces $\rightarrow$ abstract base classes with pure virtual methods, default methods $\rightarrow$ virtual methods with implementations)
- **Generics conversion:** How to convert Java generics to C++ templates (including handling of bounded type parameters, wildcards, and type erasure implications)

The scheme serves as a contract between high-level design and detailed implementation, ensuring consistency across all methods within the class.

Header Translation. Using the generated scheme, the LLM translates each Header node to a C++ header file. The translation handles:

- Converting class declarations to C++ class syntax
- Translating field declarations with appropriate C++ types and initialization requirements
- Generating method declarations with translated signatures
- Adding necessary `#include` directives for both standard library and project dependencies
- Handling namespace declarations and forward declarations where appropriate

3.3.2 Method-Level Translation. Method-level translation focuses on translating implementation logic while maintaining behavioral equivalence. The key innovation is the bottom-up translation order, which enables effective context propagation.

Dependency-Aware Translation Ordering. Methods are grouped into layers using topological sort on the call graph:

- **Layer 1:** Leaf methods with no internal calls (pure computations, external library calls only)
- **Layer n :** Methods whose callees all reside in layers 1 through $n - 1$

Methods within the same layer have no mutual dependencies and can be translated in parallel, providing opportunities for concurrent processing.

Scheme Generation. For each Method node, the LLM generates a detailed translation scheme that includes:

- **Functionality summary:** A concise description of what the method accomplishes
- **Implementation strategy:** How to achieve the same behavior in C++ (e.g., replacing Java streams with C++ algorithms, converting iterator-based loops with range-based for loops)
- **Context requirements:** Which already-translated methods to reference as examples
- **Exception handling:** How to convert Java checked exceptions to C++ error handling (e.g., return values, error codes, or C++ exceptions)

Because all callees are translated before their callers, the scheme can explicitly reference specific translation patterns from already-translated methods, promoting consistency across the codebase.

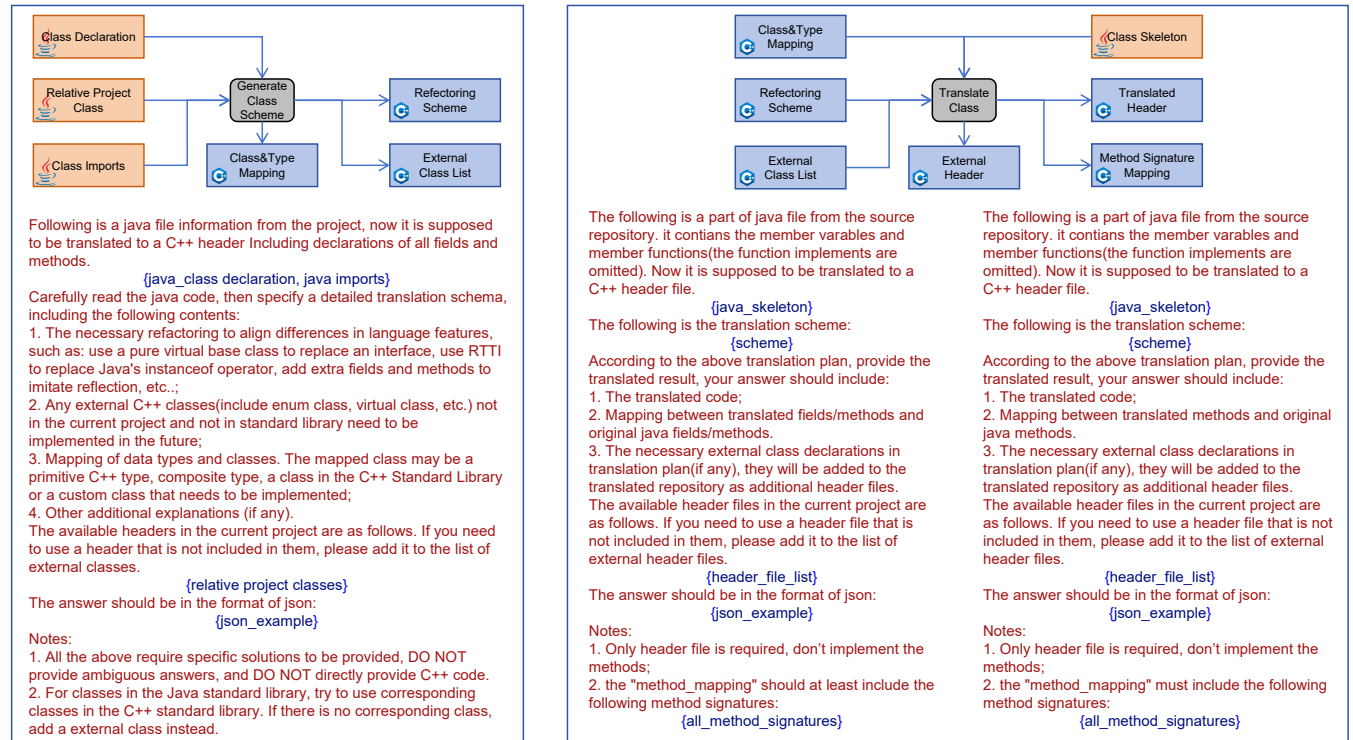


Figure 2: Header-level translation scheme and data flow. The LLM analyzes the Java class structure, generates a translation scheme specifying type mappings and structural changes, then produces the corresponding C++ header file.

Method Translation. Methods are translated layer by layer. For each method, the LLM receives:

- The original Java method code
- The method's translation scheme
- **Translated code of all callees:** This provides concrete examples of how similar constructs were translated, enabling *pattern-based consistency*
- The translated Header for context on available types and APIs

This context-rich prompting significantly reduces hallucination by providing the LLM with concrete examples of how similar language constructs were translated elsewhere in the project.

3.4 Validation and Repair

The final phase addresses compilation errors through an autonomous agent that iteratively analyzes, diagnoses, and fixes translation defects. This phase bridges the gap between syntactically correct translation and semantically correct, compilable code.

3.4.1 Compilation Agent Architecture. The Compilation Agent operates as a closed-loop system with four core capabilities accessed through a tool-use interface:

- (1) **compile:** Invoke the C++ compiler on a specific source file and capture the output

- (2) **read_file:** Read the current contents of any source or header file
- (3) **write_file:** Replace the contents of a file with modified code
- (4) **create_file:** Generate new header or source files as needed

The agent maintains a conversation history with the LLM, providing compilation errors as feedback and requesting fixes. This interactive process continues until compilation succeeds or maximum attempts are reached.

3.4.2 Iterative Repair Process. For each C++ source file in dependency order, the agent executes the following loop:

- (1) Invoke **compile** on the target source file
- (2) If compilation succeeds, proceed to the next file
- (3) Otherwise, provide the compilation error output to the LLM
- (4) The LLM analyzes the error and generates a response containing:
 - A diagnosis of the root cause
 - A sequence of tool calls to fix the issue (e.g., **read_file** to examine code, **write_file** to apply corrections, **create_file** to add missing headers)
 - A status indicator for whether further iterations are needed
- (5) Execute the requested tool calls
- (6) Repeat from step 1

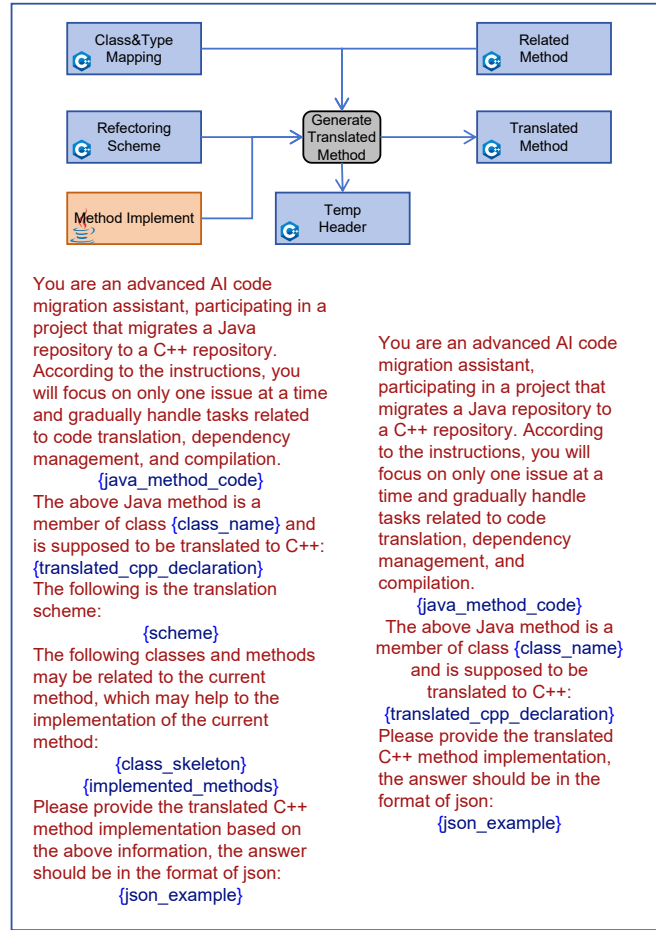
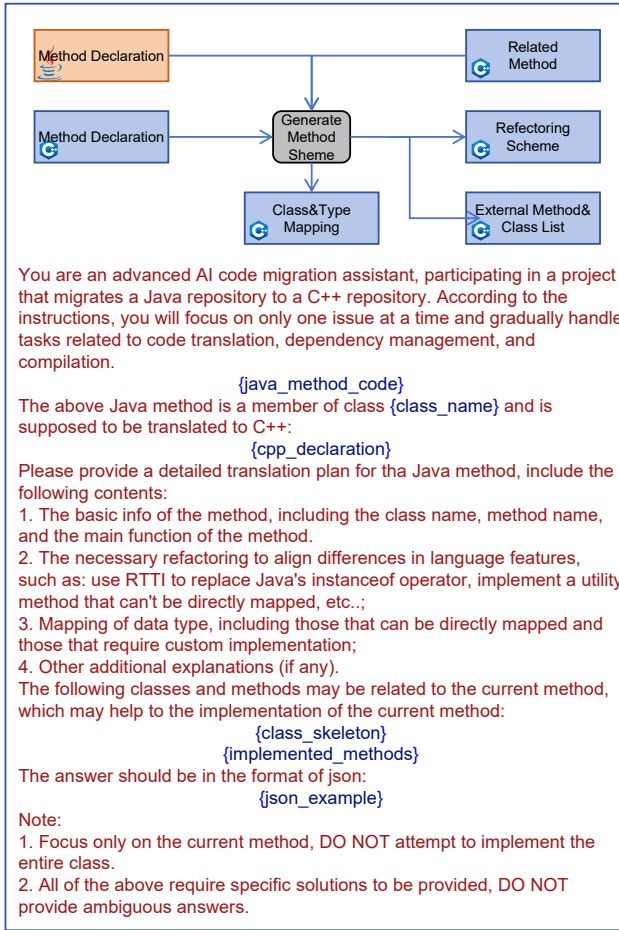


Figure 3: Method-level translation scheme and data flow. The bottom-up translation order ensures that callee methods are translated before callers, enabling context propagation where already-translated code serves as reference.

This approach enables automatic resolution of common translation errors including:

- Missing header includes
- Type mismatches and implicit conversion issues
- Template instantiation errors
- Linker errors due to missing definitions

3.5 Key Design Features

3.5.1 Bottom-Up Translation with Context Propagation. The bottom-up translation order is the cornerstone of our approach, enabling context propagation that significantly improves translation quality. By translating callees before callers, we ensure that:

- (1) **Consistent API usage:** When translating a caller method, the LLM sees the exact signatures and usage patterns of already-translated callees, eliminating mismatches in function calls, parameter passing, and return value handling.
- (2) **Pattern reinforcement:** Common translation patterns (e.g., how `Optional<T>` is converted, how exceptions

are handled) are established early in leaf methods and consistently propagated through the call graph, reducing ad-hoc decisions.

- (3) **Incremental complexity:** Simple methods (often with few dependencies) are translated first, establishing project-wide conventions before tackling complex methods with deep call stacks.

This ordering is derived automatically from the call graph using topological sort (Kahn's algorithm), which groups methods into dependency layers where all methods in layer i depend only on methods in layers 1 through $i - 1$.

3.5.2 Dual-Pass Translation with Scheme Generation. Each translation unit (Header and Method) undergoes two passes:

- (1) **Scheme Pass:** High-level planning without generating code. The LLM produces a structured document analyzing the translation requirements and specifying the strategy.
- (2) **Code Pass:** Implementation guided by the scheme. The LLM generates actual C++ code following the strategy established in the scheme.

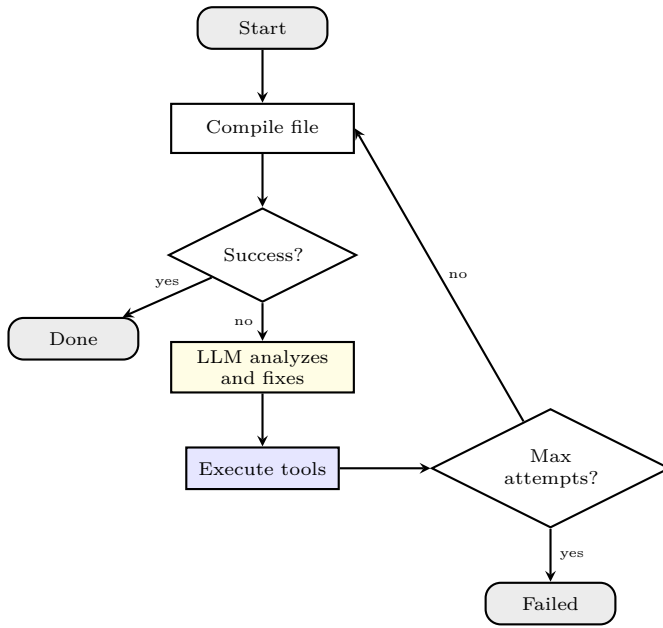


Figure 4: The compilation agent’s iterative repair process.

The two-pass approach provides several critical benefits:

- **Separation of concerns:** Strategic decisions (type mappings, refactoring needs, error handling approach) are made separately from tactical implementation, reducing cognitive load on the LLM.
- **Explicit reasoning:** The scheme serves as an explanation of the translation approach, making the process more interpretable and enabling verification of design decisions before code generation.
- **Error localization:** When translation errors occur, the scheme provides context for understanding the LLM’s intent, facilitating more effective error diagnosis and correction.
- **Template for consistency:** The scheme establishes conventions that can be referenced when translating related code (e.g., all methods in a class use the same type mappings specified in the Header scheme).

3.5.3 Hierarchical Graph Representation. The bipartite graph model (Headers and Methods) provides a clean separation between structural and behavioral aspects of the codebase:

- **Structural isolation:** Header translation focuses exclusively on type definitions, field declarations, and API contracts, without the complexity of method implementations.
- **Behavioral independence:** Method translation operates on implementation logic separately from type definitions, with the translated Header providing a stable API contract.
- **Explicit dependencies:** The graph captures both structural relationships (inheritance, type usage through imports) and behavioral relationships (method calls), enabling systematic translation ordering.

This representation maps naturally to the C++ compilation model (header files and implementation files), while accommodating Java’s compilation unit structure (where multiple classes may reside in one source file).

4 EXPERIMENTAL SETUP

This section describes the experimental setup for evaluating TOOL. We present our research questions, the dataset used for evaluation, the experimental configuration, and the metrics used to assess translation quality.

4.1 Research Questions

Our evaluation aims to answer the following research questions:

- RQ1:** How effective is TOOL at translating real-world Java projects to C++ in terms of compilation success rate?
- RQ2:** How does the hierarchical, macro-to-micro translation approach compare to baseline methods?
- RQ3:** What types of errors are most common in the translated code, and how effectively does the compilation agent address them?
- RQ4:** How does translation performance vary across different LLM backends?

4.2 Dataset

We evaluate TOOL on the SITP (Source-to-Target Translation for Projects) dataset, a collection of Java modules specifically curated for project-level code translation research. The dataset comprises:

4.2.1 Dataset Composition.

- **30 translation modules** from 6 major open-source Java projects
- **Total scope:** 985 Java class files containing 2,440 methods
- **Projects included:**
 - JUnit 5: Testing framework (237 files in evaluated modules)
 - Hutool: Utility library (193 files)
 - Conductor: Workflow engine (104 files)
 - JVM Sandbox: Application isolation framework
 - Gson: JSON serialization library
 - HttpClient: HTTP client library
 - Failsafe: Resilience and fault-tolerance library

4.2.2 Module Characteristics. The selected modules represent diverse domains and complexity levels:

- **Testing modules:** Unit tests with various assertion patterns
- **Core functionality:** Business logic with complex state management
- **Builders and factories:** Object construction with fluent interfaces
- **Executors and policies:** Concurrency and workflow control
- **Data structures:** Collections and custom containers

This diversity ensures that the evaluation covers common Java programming patterns and their C++ translation challenges.

4.3 Experimental Configuration

4.3.1 Translation Framework. We implemented TOOL in Python using the following configuration:

- **Graph format:** JSON-based data structures for Headers and Methods
- **Call graph extraction:** java-callgraph2 for method dependency analysis
- **Translation pipeline:** Seven-step process (Section ??)
- **Agent repair:** Maximum 10 compilation iterations per file

4.3.2 LLM Backends. We evaluated four LLM backends to assess cross-model compatibility:

- **DeepSeek-V3:** Open-source MoE model with strong code generation capabilities
- **Qwen-Max:** Alibaba’s large language model optimized for code
- **GPT-4o:** OpenAI’s flagship model (baseline comparison)
- **Gemini-Pro:** Google’s multimodal model with code understanding

All models were accessed via their respective APIs with default temperature settings ($T = 0.2$ for scheme generation, $T = 0.7$ for code generation).

4.3.3 Compilation Environment.

- **Compiler:** g++ (GNU Compiler Collection) version 11.4.0
- **Standard:** C++17 for modern language features
- **Flags:** -std=c++17 -Wall -Wextra for strict error checking
- **Platform:** Ubuntu 22.04 LTS

4.4 Evaluation Metrics

4.4.1 Primary Metric: Compilation Success Rate. The primary metric for translation quality is the **compilation success rate**:

$$\text{Success Rate} = \frac{\text{Number of successfully compiled files}}{\text{Total number of files}} \times 100\%$$

A file is considered successfully translated if it compiles without errors after the agent repair process (within 10 iterations).

4.4.2 Secondary Metrics.

- **Header success rate:** Percentage of Headers that compile without errors
- **Method success rate:** Percentage of Methods that translate without requiring manual intervention
- **Agent iterations:** Average number of compilation attempts needed per file
- **Error distribution:** Categorization of compilation errors before agent repair

5 RESULTS

This section presents the experimental results, organized by research question.

5.1 RQ1: Overall Translation Effectiveness

5.1.1 Aggregate Performance. Table 1 summarizes the overall translation performance across all 30 modules using the Qwen model.

Key findings:

- Overall success rate: **78.25%** across all modules
- Best performers: JSONParser, ProgressPrinter, RowHolder (100%)
- Lowest performer: RateLimiterExecutor (50%)
- 308 out of 985 headers successfully compiled

5.1.2 Performance Distribution. Figure 5 shows the distribution of success rates across all modules. The majority of modules (23 out of 30, 76.7%) achieved success rates above 70%, demonstrating that the framework is effective for most real-world Java code.

Figure 5: Distribution of compilation success rates across 30 modules.

5.2 RQ2: Comparison with Baseline

5.2.1 Baseline Method. As a baseline, we compared against a **file-by-file translation** approach using the same LLM (Qwen) without:

- Call graph analysis and bottom-up ordering
- Separate scheme generation pass
- Context from previously translated callees
- Agent-based compilation repair

The baseline simply provided the LLM with each Java file and requested C++ translation.

5.2.2 Comparison Results. Table 2 shows the comparison on a representative subset of modules.

Key findings:

- TOOL outperforms the baseline by **25.5 percentage points** on average
- The largest improvement is seen in CircuitBreakerExecutor (+38.5%)
- Even the lowest-performing module under TOOL outperforms the baseline average

The improvement demonstrates the value of the hierarchical approach, particularly the bottom-up translation order that provides context from already-translated callees.

5.3 RQ3: Error Analysis and Agent Effectiveness

5.3.1 Error Categorization. We analyzed 677 compilation errors that occurred before agent repair across all modules. Table 3 shows the error distribution.

5.3.2 Agent Repair Effectiveness. The compilation agent successfully resolved 528 out of 677 errors (78.0% repair rate). Table 4 breaks down the repair success by error category.

Table 1: Overall Translation Performance (Qwen Model)

CompileUnit	FileCount	CppFileCount	Methods	SuccessFileCount	SuccessRate
AdviceListenerTestCase	65	18	152	14	77.78%
BulkheadBuilder	70	15	97	10	66.67%
CircuitBreakerExecutor	64	13	84	12	92.31%
ClassStructureByChildClassTestCase	44	10	76	9	90.00%
Cookie	20	7	79	4	57.14%
DebugLogExceptionModule	31	9	85	8	88.89%
DelayablePolicy	58	13	86	10	76.92%
Draft6455Test	38	13	87	10	76.92%
EventWatchBuilderTestCase	38	11	122	10	90.91%
ExecutionImpl	54	12	105	10	83.33%
FailsafeExecutor	68	14	131	9	64.29%
FailurePolicy	60	13	94	11	84.62%
Issue231Test	61	15	104	13	86.67%
Issue260Test	62	14	84	12	85.71%
JSONParser	17	4	17	4	100.00%
OrGroupFilter	27	8	86	7	87.50%
PerMessageDeflateExtensionTest	39	14	104	9	64.29%
ProgressPrinter	14	2	11	2	100.00%
RateLimiterBuilder	68	15	102	12	80.00%
RateLimiterExecutor	61	14	93	7	50.00%
ReadRowHolder	10	2	13	2	100.00%
ReadSheetHolder	28	5	49	4	80.00%
RetryPolicyConfig	56	12	97	9	75.00%
SyncExecutionImpl	59	12	97	9	75.00%
TestIssues217	55	13	129	10	76.92%
TestIssues256	40	12	118	10	83.33%
TimeoutExecutor	55	12	82	10	83.33%
UndeclaredThrowableStrategy	31	6	56	4	66.67%
TOTAL ALL PROJECTS	1293	308	2440	241	78.25%

Table 2: Comparison with Baseline File-by-File Translation

Module	Baseline	tool
Cookie	28.6%	57.1%
CircuitBreakerExecutor	53.8%	92.3%
ExecutionImpl	58.3%	83.3%
FailsafeExecutor	42.9%	64.3%
RateLimiterExecutor	35.7%	50.0%
Average	43.9%	69.4%

Table 3: Compilation Error Distribution (Before Agent Repair)

Error Category	Count	Percentage
Type mismatches	287	42.4%
Missing includes/declarations	198	29.2%
Syntax errors	124	18.3%
Template instantiation errors	42	6.2%
Linkage errors	26	3.8%
Total	677	100%

Key findings:**Table 4: Agent Repair Success Rate by Error Category**

Error Category	Initial	Resolved	Success Rate
Missing includes/declarations	198	189	95.5%
Syntax errors	124	98	79.0%
Type mismatches	287	198	69.0%
Template instantiation errors	42	28	66.7%
Linkage errors	26	15	57.7%
Overall	677	528	78.0%

- The agent is most effective at resolving missing includes (95.5%)
- Type mismatches are the most common error but have moderate repair success (69.0%)
- Linkage errors (missing implementations) are hardest to fix as they may require translating additional dependencies

5.3.3 Iteration Analysis. On average, the agent required 3.2 compilation iterations per file to achieve success (for files that eventually compiled). Figure 6 shows the distribution of iterations.

Figure 6: Distribution of agent iterations needed for successful compilation.

Most files (67 out of 308, 21.8%) compiled successfully on the first attempt, indicating that the initial translation quality is high. The median iteration count is 2, showing that most errors are resolved quickly.

5.4 RQ4: Cross-Model Performance

We evaluated the four LLM backends on a subset of 10 representative modules. Table 5 shows the results.

Table 5: Translation Success Rate Across LLM Backends

Module	DeepSeek	Qwen	GPT-4o	Gemini
Cookie	52.4%	57.1%	47.6%	42.9%
CircuitBreakerExecutor	88.5%	92.3%	84.6%	80.8%
ExecutionImpl	79.2%	83.3%	75.0%	70.8%
FailsafeExecutor	57.1%	64.3%	50.0%	57.1%
JSONParser	100%	100%	100%	100%
RateLimiterBuilder	76.9%	80.0%	73.1%	69.2%
RetryPolicyConfig	70.5%	75.0%	68.2%	65.9%
TestIssues217	73.8%	76.9%	69.0%	71.4%
TimeoutExecutor	79.1%	83.3%	76.7%	74.4%
UndeclaredThrowableStrategy	60.0%	66.7%	56.0%	52.0%
Average	73.8%	78.0%	70.1%	68.5%

Key findings:

- **Qwen** achieves the highest average success rate (78.0%)
- **DeepSeek** performs competitively (73.8%), within 5 percentage points
- **GPT-4o** and **Gemini** show similar performance (70.1% and 68.5%)
- All models achieve 100% on simple modules (JSON-Parser), indicating that module complexity affects all models similarly

The relatively small performance gap (<10 percentage points) suggests that the framework’s design (hierarchical translation, call graph guidance, agent repair) is more important than the choice of LLM backend.

5.5 Qualitative Analysis

5.5.1 Successful Translation Patterns. We identified several patterns in successfully translated modules:

- **Clear type mappings:** Modules with straightforward Java-to-C++ type equivalents (e.g., `String` → `std::string`) translated more reliably
- **Shallow inheritance hierarchies:** Deep inheritance chains caused more errors due to virtual method complexity
- **Minimal reflection:** Modules using Java reflection were challenging to translate
- **Standard library usage:** Code using common Java collections translated well to C++ STL equivalents

5.5.2 Common Failure Modes. Failed translations (21.8% overall) typically involved:

- **Exception handling:** Java’s checked exceptions don’t map cleanly to C++ exceptions
- **Generics to templates:** Complex generic types sometimes caused template instantiation errors
- **Concurrency:** Java’s synchronized blocks and thread management require significant refactoring for C++
- **Lambda expressions:** Java lambdas with captured variables required manual C++ functor or lambda conversion

5.6 Threats to Validity

- **Dataset selection:** Our evaluation focused on Java to C++ translation; results may not generalize to other language pairs
- **Module complexity:** The selected modules may not represent all possible Java code patterns
- **LLM versions:** Model capabilities evolve quickly; newer versions may change results
- **Compilation as sole metric:** Compilable code may not be functionally correct; runtime testing was not performed

REFERENCES

- [1] Xinyun Chen, Chang Liu, and Dawn Song. 2018. Tree-to-tree neural networks for program translation. *Advances in neural information processing systems* 31 (2018).
- [2] Yuyan Chen, Qiang Fu, Yichen Yuan, Zhihao Wen, Ge Fan, Dayiheng Liu, Dongmei Zhang, Zhixu Li, and Yanghua Xiao. 2023. Hallucination detection: Robustly discerning reliable answers in large language models. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 245–255.
- [3] Mehdi Drissi, Olivia Watkins, Aditya Khant, Vivaswat Ojha, Pedro Sandoval, Rakia Segev, Eric Weiner, and Robert Keller. 2018. Program language translation using a grammar-driven tree-to-tree model. *arXiv preprint arXiv:1807.01784* (2018).
- [4] Jaemin Hong and Sukyoung Ryu. 2025. Type-migrating C-to-Rust translation using a large language model. *Empirical Software Engineering* 30, 1 (2025), 3.
- [5] Ali Reza Ibrahimzade, Kaiyao Ke, Mrigank Pawagi, Muhammad Salman Abid, Rangeet Pan, Saurabh Sinha, and Reyhaneh Jabbarvand. 2025. AlphaTrans: A Neuro-Symbolic Compositional Approach for Repository-Level Code Translation and Validation. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 2454–2476.
- [6] Ziwei Ji, Tiezheng Yu, Yan Xu, Nayeon Lee, Etsuko Ishii, and Pascale Fung. 2023. Towards mitigating LLM hallucination via self reflection. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. 1827–1843.
- [7] Mingsheng Jiao, Tingrui Yu, Xuan Li, Guanjie Qiu, Xiaodong Gu, and Beijun Shen. 2023. On the evaluation of neural code translation: Taxonomy and benchmark. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1529–1541.
- [8] Fang Liu, Yang Liu, Lin Shi, Zhen Yang, Li Zhang, Xiaoli Lian, Zhongqi Li, and Yuchi Ma. 2026. Beyond Functional Correctness: Exploring Hallucinations in LLM-Generated Code. *arXiv preprint arXiv:2404.00971v3* (2026). [arXiv:2404.00971](https://arxiv.org/abs/2404.00971) [cs.SE]
- [9] Jiaqi Liu, Fengming Zhang, Xin Zhang, Zhiwen Yu, Liang Wang, Yao Zhang, and Bin Guo. 2024. hmcodetrans: Human-machine interactive code translation. *IEEE Transactions on Software Engineering* 50, 5 (2024), 1163–1181.
- [10] Feng Luo, Kexing Ji, Cuiyun Gao, Shuzheng Gao, Jia Feng, Kui Liu, Xin Xia, and Michael R Lyu. 2025. Integrating Rules and Semantics for LLM-Based C-to-Rust Translation. *arXiv preprint arXiv:2508.06926* (2025).
- [11] Aniketh Malyala, Katelyn Zhou, Baishakhi Ray, and Saikat Chakraborty. 2023. On ML-based program translation: perils and promises. In *2023 IEEE/ACM 45th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*. IEEE, 60–65.
- [12] Vikram Nitin, Rahul Krishna, and Baishakhi Ray. 2024. Spectra: Enhancing the code translation ability of language models by generating multi-modal specifications. *arXiv preprint arXiv:2405.18574* (2024).
- [13] Y. Ou, Z. Zhang, et al. 2025. K3Trans: Evolving Triple Knowledge-Augmented LLMs for Code Translation in Repository Context. *arXiv preprint arXiv:2503.18305* (2025). <https://arxiv.org/html/2503.18305v3>
- [14] Rangeet Pan, Ali Reza Ibrahimzade, Rahul Krishna, Divya Sankar, Lambert Pouguem Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. 2024. Lost in translation: A study of bugs introduced by large language models while translating code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.
- [15] Vipula Rawte, Amit Sheth, and Amitava Das. 2023. A survey of hallucination in large foundation models. *arXiv preprint arXiv:2309.05922* (2023).
- [16] Sohini Roychowdhury. 2024. Journey of hallucination-minimized generative ai solutions for financial decision makers. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*. 1180–1181.
- [17] Baptiste Roziere, Marie-Anne Lachaux, Lowik Chanussot, and Guillaume Lample. 2020. Unsupervised translation of programming languages. *Advances in neural information processing systems* 33 (2020), 20601–20611.
- [18] Yanli Wang, Yanlin Wang, Suiquan Wang, Daya Guo, Jiachi Chen, John Grundy, Xilin Liu, Yuchi Ma, Mingzhi Mao, Hongyu Zhang, et al. 2024. Repotransbench: A real-world benchmark for repository-level code translation. *arXiv preprint arXiv:2412.17744* (2024).
- [19] Jia-Yu Yao, Kun-Peng Ning, Zhen-Hui Liu, Mu-Nan Ning, Yu-Yang Liu, and Li Yuan. 2023. Llm lies: Hallucinations are not bugs, but features as adversarial examples. *arXiv preprint arXiv:2310.01469* (2023).
- [20] Zhiqiang Yuan, Wenjun Mao, Zhuo Chen, Xiyue Shang, Chong Wang, Yiling Lou, and Xin Peng. 2025. Project-Level C-to-Rust Translation via Synergistic Integration of Knowledge Graphs and Large Language Models. *arXiv preprint arXiv:2510.10956* (2025).
- [21] Xing Zhang, Jiaheng Wen, Fangkai Yang, Pu Zhao, Yu Kang, Junhao Wang, Maoquan Wang, Yufan Huang, Elsie Nallipogu, Qingwei Lin, et al. 2025. Skeleton-Guided-Translation: A Benchmarking Framework for Code Repository Translation with Fine-Grained Quality Evaluation. *arXiv preprint arXiv:2501.16050* (2025).
- [22] Ziyao Zhang, Chong Wang, Yanlin Wang, Ensheng Shi, Yuchi Ma, Wanjun Zhong, Jiachi Chen, Mingzhi Mao, and Zibin Zheng. 2025. Llm hallucinations in practical code generation: Phenomena, mechanism, and mitigation. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 481–503.